Reconstructing Chrome Web History

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

11th October 2026

LAB OVERVIEW

In this lab, I went through a database of activities carried out using SQLite3 as an interactive tool to examine and analyse the Chrome browser history. I was able to recover and review 74 web activity records from April 19, 2022, and establish the sequence of activities carried out within the browser.

The investigation involved tracing web visits, identifying email communications, locating a payment receipt file, and verifying a transaction hash across public blockchain explorers and the Kraken exchange platform. This helped me understand how browser history and SQLite databases can be used in digital forensics to reconstruct a user's browsing activities and establish a timeline of events.

I also learned the importance of interpreting forensic evidence carefully, as browser activity can establish what happened within a browser environment but cannot, on its own, conclusively identify the person responsible for the activities.

OBJECTIVES

- To preserve the supplied Chrome History database as evidence by recording its source, size and SHA-256 hash, and analysing only a working copy.
- To examine the database structure and use SQL queries on the urls, visits and downloads tables to extract and convert WebKit timestamps to UTC.
- To interpret transition values, qualifiers and from_visit links in order to reconstruct how the browser reached each page (typed, link, form submission, redirect).
- To build an evidence-based timeline of the suspected drug-trafficking activity (posting, communication, payment, download) and give a cautious conclusion that separates browser activity from a person’s identity.

TOOLS

- Kali Linux (VM in VMware Workstation)
- sqlite3 (command line)
- DB Browser for SQLite (optional, for viewing tables and screenshots)
- sha256sum
- stat and file
- Linux shell utilities (cp, chmod, unzip, find, tee, date)
- Course slides (reconstruct_webhistory.pptx) and the case study brief as reference material

METHODOLOGY

Lab Folder Structure and Evidence Preparation

For this case study we first have to create a lab folder where all the lab work would be done. We then create a Lab Folder named **SBT-DF204-CS2.**

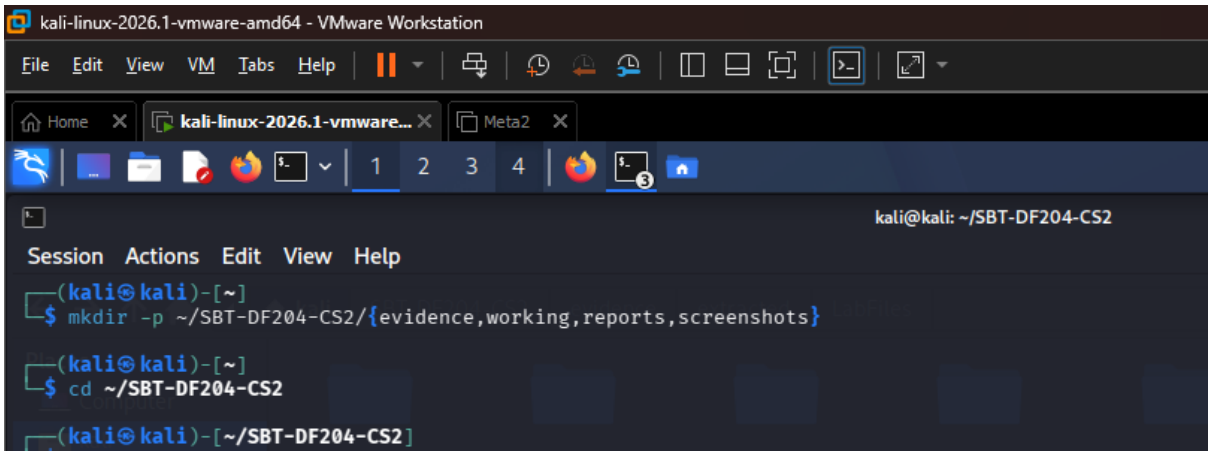


Fig 1.1 Created Lab Folder/Environment

The Folder included other folders that would be necessary for the purpose of this lab. Then we have to update the Kali environment and install necessary tools to carry out the forensic analysis.

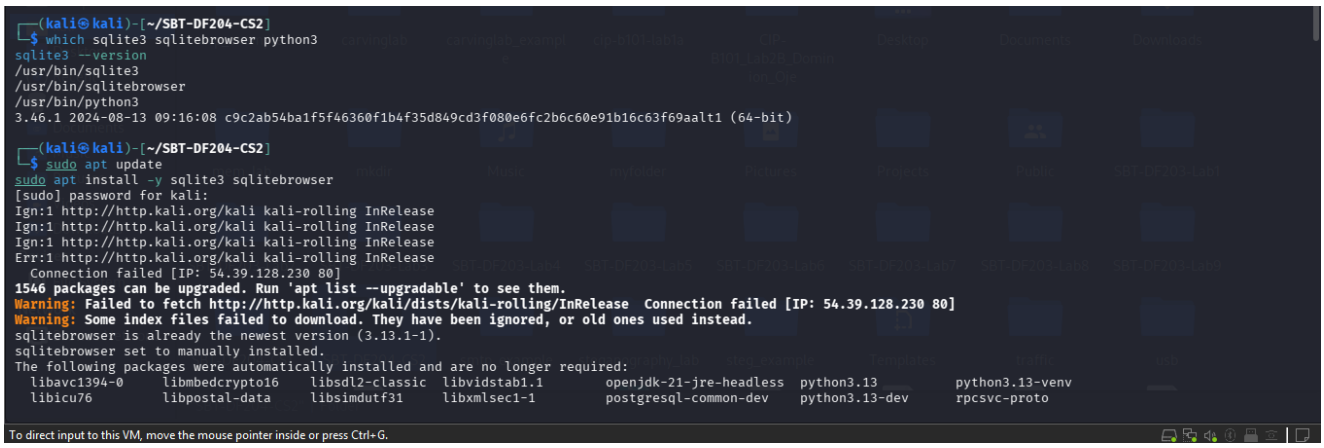


Fig 1.2 Downloaded Tools and updated Environment

Then we download the evidence file and it was downloaded on windows and it was moved into the working folder in the kali environment and now would confirm the presence of the zip file in the Lab folder. And then we used the stat command with some options to get the the file size for record purpose and to maintain integrity.

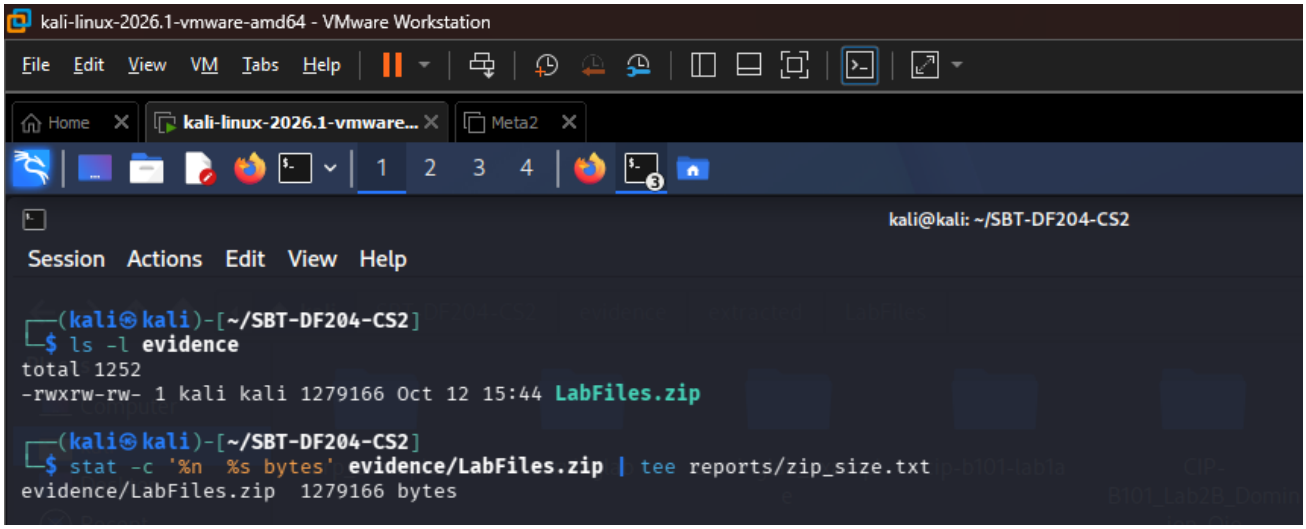


Fig 1.3 Evidence Successfully in the Lab Folder

Now to preserve the integrity of the Main evidence we get the hash value first and then we get a working copy and then we check the hash value again after that.



Fig 1.4 The shsa256sum value of the Zipped Evidence file

Then we make the change the file permission of the zipped folder so we can unzip it and get the files we would need for this lab.

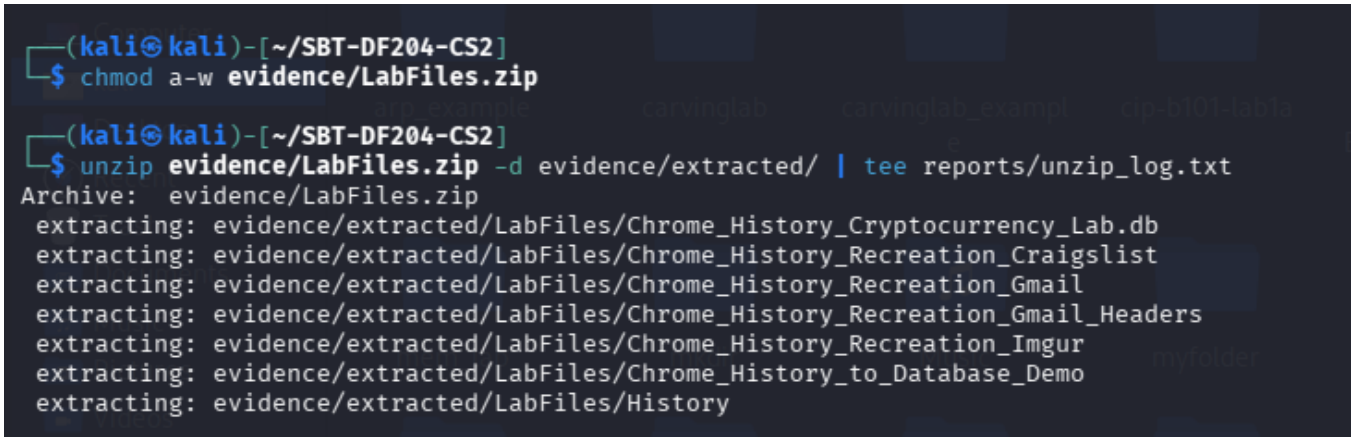


Fig 1.5 Extracted Files from the Zipped folder

Now we want to check the hash value of all the extracted files which are now in a folder named extracted which was created in the last command.

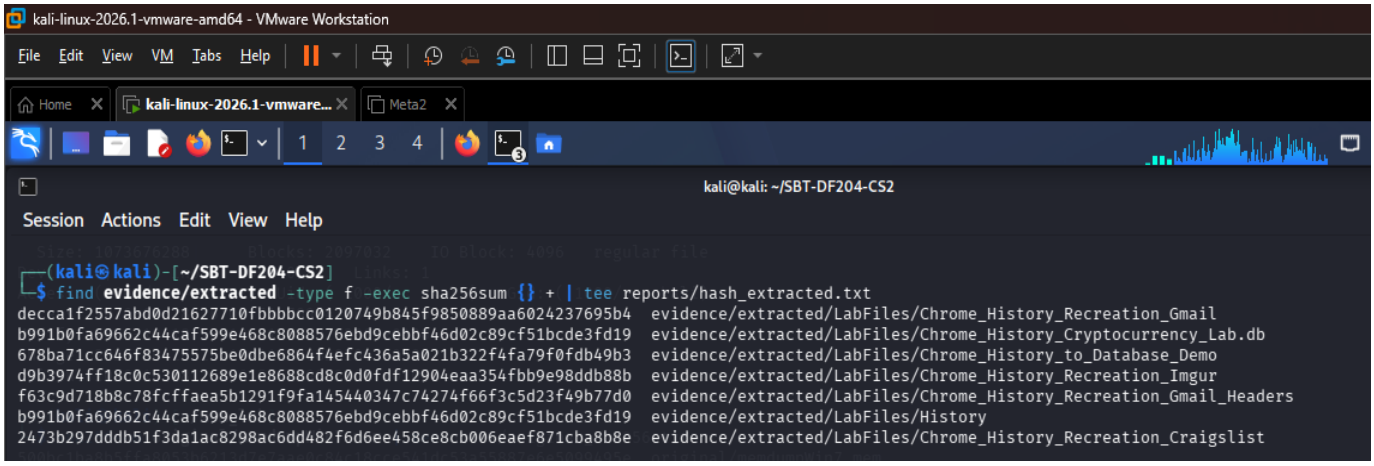


Fig 1.6 Hashed values of all the files in the extracted folder

From Figure 1.6 we can see that both files **Chrome_History_Cryptocurrency_Lab.db** and **History** has the same hash value of **b991b0fa69662c44caf599e468c8088576ebd9cebbf46d02c89cf51bcde3fd19** which depicts that they have the same content and the same byte size and are identical.

Now we focus on the File that we would be using to carry out our analysis which from the Case Folder it is the History file. We then make a working copy of the History File so we can carry analysis on it and keep the preserve the evidence file.



Fig 1.7 Preserve the main History File by copying it

Now we check the hash values of both the history file and the copied working file to see the integrity of the working file.

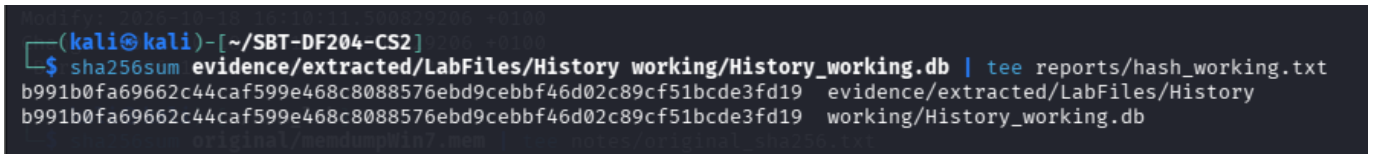


Fig 1.8 Hash Values of both file

From Fig 8 we can see that both the main History file in the evidence folder and the working history file from the working folder has both the same hash value. Now we can begin the analysis of the Working file.



Fig 1.9 Checking the Byte of the File

Lastly, we check the details of the working History file using file



Fig 1.10 Details of the Working File

DATABASE SCHEMA INSPECTION

The next step is to study the layout or schema of the data base. I then, opened the working copy inside the interactive SQLite command-line tool directly from my Kali Linux terminal. Before running any analysis queries, I opened the working copy of the History database in read-only mode (sqlite3 -readonly), so the evidence could not be altered. I listed the tables and inspected the schema of each one I intended to use.

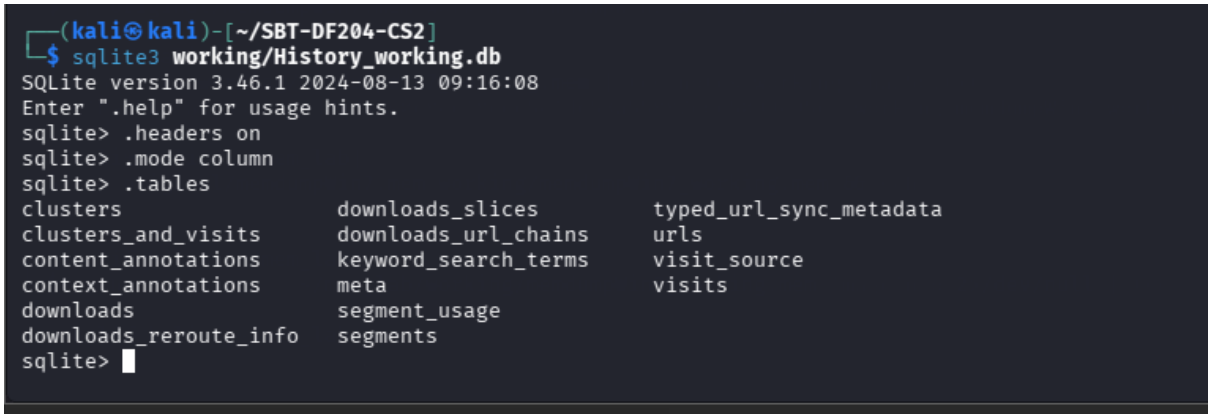


Fig 1.11 Tables present in the working copy of the History

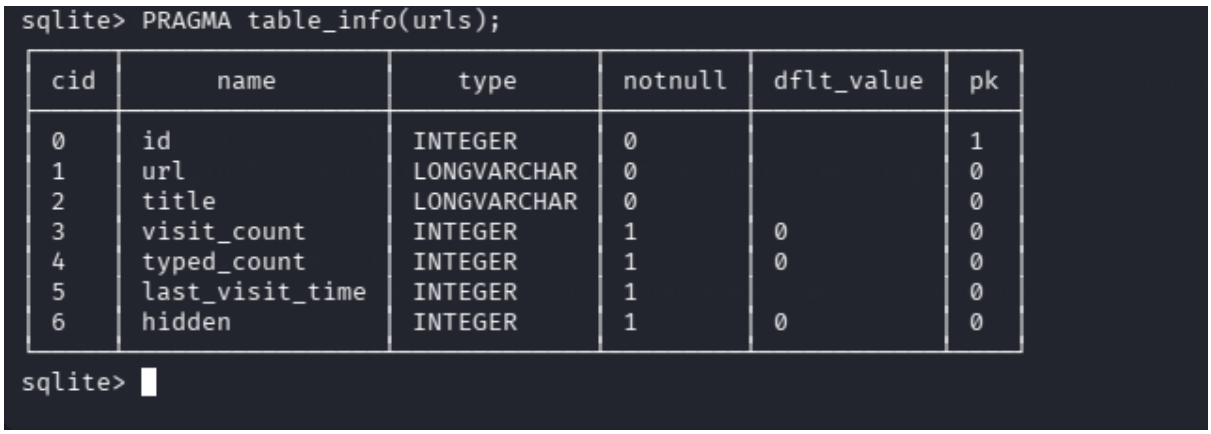


Fig 1.12 Schema of the urls table

```
sqlite> PRAGMA table_info(visits)
   ...> ;
```

cid	name	type	notnull	dflt_value	pk
0	id	INTEGER	0		1
1	url	INTEGER	1		0
2	visit_time	INTEGER	1		0
3	from_visit	INTEGER	0		0
4	transition	INTEGER	1	0	0
5	segment_id	INTEGER	0		0
6	visit_duration	INTEGER	1	0	0
7	incremented_omnibox_typed_score	BOOLEAN	1	FALSE	0
8	opener_visit	INTEGER	0		0

```
sqlite>
```

Fig 1.13 Schema of the visits table

```
sqlite> PRAGMA table_info(downloads);
```

cid	name	type	notnull	dflt_value	pk
0	id	INTEGER	0		1
1	guid	VARCHAR	1		0
2	current_path	LONGVARCHAR	1		0
3	target_path	LONGVARCHAR	1		0
4	start_time	INTEGER	1		0
5	received_bytes	INTEGER	1		0
6	total_bytes	INTEGER	1		0
7	state	INTEGER	1		0
8	danger_type	INTEGER	1		0
9	interrupt_reason	INTEGER	1		0
10	hash	BLOB	1		0
11	end_time	INTEGER	1		0
12	opened	INTEGER	1		0
13	last_access_time	INTEGER	1		0
14	transient	INTEGER	1		0
15	referrer	VARCHAR	1		0
16	site_url	VARCHAR	1		0
17	tab_url	VARCHAR	1		0
18	tab_referrer_url	VARCHAR	1		0
19	http_method	VARCHAR	1		0
20	by_ext_id	VARCHAR	1		0
21	by_ext_name	VARCHAR	1		0
22	etag	VARCHAR	1		0
23	last_modified	VARCHAR	1		0
24	mime_type	VARCHAR(255)	1		0
25	original_mime_type	VARCHAR(255)	1		0
26	embedder_download_data	VARCHAR	1	''	0

```
sqlite>
```

Fig 1.14 Schema of the downloads table

```
sqlite> PRAGMA table_info(downloads_url_chains);
```

cid	name	type	notnull	dflt_value	pk
0	id	INTEGER	1		1
1	chain_index	INTEGER	1		2
2	url	LONGVARCHAR	1		0

```
sqlite>
```

Fig 1.15 Schema of the downloads_url_chains table

```
sqlite> PRAGMA table_info(keyword_search_terms);
```

cid	name	type	notnull	dflt_value	pk
0	keyword_id	INTEGER	1		0
1	url_id	INTEGER	1		0
2	term	LONGVARCHAR	1		0
3	normalized_term	LONGVARCHAR	1		0

```
sqlite>
```

Fig 1.16 Schema of the keyword_search_terms

The analysis rests on five tables:

- **urls** holds one row for each distinct address, with its title, visit count and last visit time.
- **visits** holds one row for each individual visit. visits.url points to urls.id, so joining the two gives the address, title and time of every visit. The transition field records how the page was reached, and from_visit links a visit to the earlier visit that led to it.
- **downloads** records each saved file, including its save path, size, start and end time, referrer and source tab.
- **downloads_url_chains** records the chain of URLs, including redirects, behind each download, and links to downloads through the download ID.
- keyword_search_terms, which records what was typed into the address bar and links back to urls.

In the process of going through the schema table I examined the urls, visits, downloads, downloads_url_chains and keyword_search_terms tables of the working copy of the History database.

The **urls** table holds one row for each distinct address, with its title, visit_count, typed_count and last_visit_time. In this case it identifies which sites the browser reached, and the titles help show what each page was.

The **visits** table holds one row for each individual visit. It links to urls through url, and supplies visit_time, transition, from_visit, opener_visit and visit_duration. It carries the

timeline: when each page was opened, how it was reached, which visit led to it, and how long it stayed open.

The **downloads** table records saved files, including target_path, start_time, end_time, total_bytes, referrer and tab_url. In this case it identifies the file saved to the computer, when it was saved, and the page it came from.

The **downloads_url_chains** table links each download ID to the chain of URLs, including redirects, behind the file. It shows the exact address the file was fetched from, which supports the link between the download and the page that preceded it.

Together, visits joined to urls gives the timeline of activity, downloads and its URL chain tie the saved file to the browsing session, and keyword_search_terms shows what was typed into the address bar.

WEB HISTORY QUERY EXECUTION

To reconstruct the chronological sequence of user activity recorded in the Chrome browser history, I ran a joined SQL query on the working copy of the History database, using the interactive sqlite3 command line in Kali Linux. The visits table records when each page was opened, but the page addresses and titles are held in the URLs table, so I joined the two on visits.url = urls.id and ordered the results by visit_time. This produced one row for each of the 74 recorded visits; in the order they happened in the figure below.

I incorporated a standard conversion formula directly into the SQL statement, which adjusted the WebKit microsecond values stored since the year 1601 into human-readable UTC date and time strings. Since chrome stores time as WebKit timestamps, which count microseconds from 1 January 1601 UTC, so the raw values cannot be read directly then I had to convert the timestamp from **Unix epoch** to UTC readable time by by dividing by 1,000,000 to get seconds and subtracting 11,644,473,600, the number of seconds between 1601 and the Unix epoch of 1970.

Also, for readability, I displayed the output in column mode with fixed widths and shortened titles and URLs. The full values remain in the database. I also kept from_visit, opener_visit and transition in the output, because these show how the user moved from one page to the next, which I analysed in the next stage.

The recorded activity runs from 13:54:18 UTC to 15:02:56 UTC on 19 April 2022, a span of about 1 hour 9 minutes from fig 1.17 and 1.18. All times in this report are UTC.

with

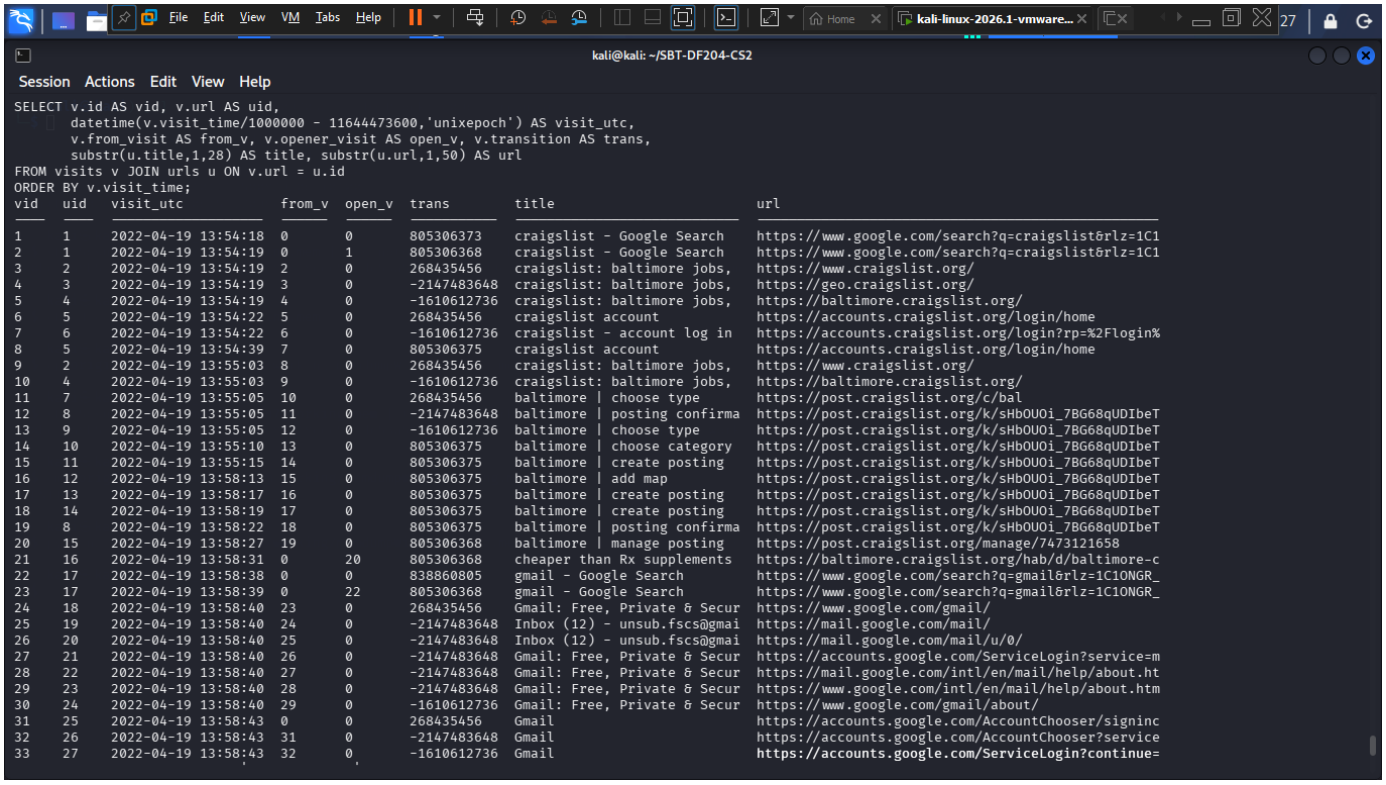


Fig 1.17 Web Activity of user with visit url

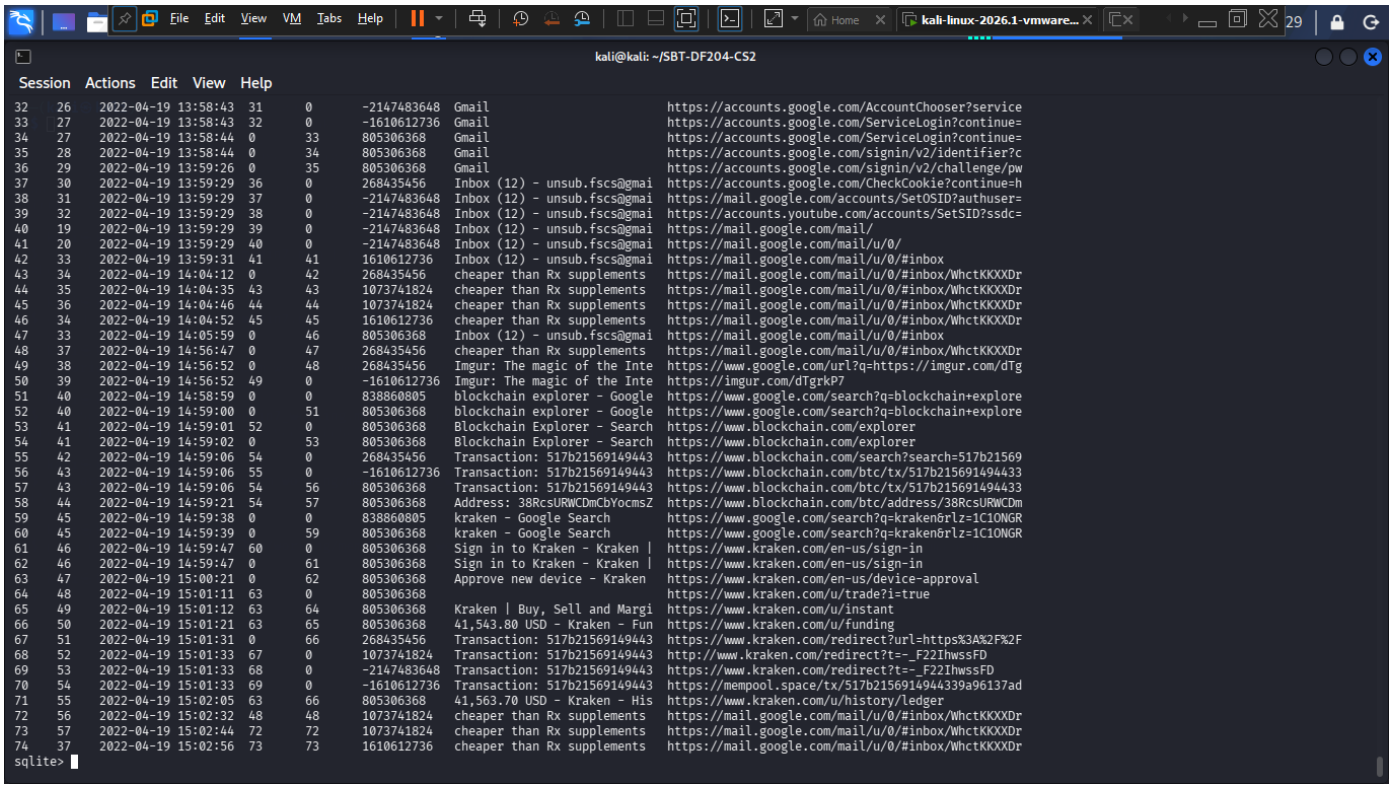


Fig 1.18 Web Activity with Duration

To determine how the browser reached each material page, I decoded the transition field, where the low byte gives the base type and the higher bits give qualifiers such as chain start and end, server redirect and client redirect, and I followed the from_visit and opener_visit links between visits.

The decoded values show that no address was typed directly. The only address-bar activity was four Google searches, at visits 1, 22, 51 and 59, for “craigslist”, “gmail”, “blockchain explorer” and “kraken”. From each set of results the user clicked through by link. Seven form submissions (0x30000007) account for the Craigslist login at visit 8 and the posting workflow at visits 14 to 19.

Several pages were reached through redirect chains. Visits 3 to 5 are one chain from a single click, with a start, a server redirect and an end. The Kraken redirect at visits 67 to 70 ended at a transaction page on mempool.space. Gmail used client redirects (0x40000000) to append compose parameters to a message address, which shows that reply windows were opened inside the thread (visits 44, 45, 72 and 73).

Where **from_visit** was 0, I used **opener_visit** to identify the page that led to the visit. For example, the live advert (visit 21) was opened from the “manage posting” page (visit 20), and the Imgur link (visit 49) was opened from an email thread (visit 48), supported by the source=gmail parameter in its address. These fields show how the browser moved between pages. They don’t show what the user did on a page, because the History database stores no page content or keystrokes.

SUSPECTED SALE AND POSTING WORKFLOW

```
sqlite> SELECT v.id AS vid, datetime(v.visit_time/1000000 - 11644473600,'unixepoch') AS visit_utc,
       round(v.visit_duration/1000000.0,1) AS secs,
       printf('%X', v.transition & 4294967295) AS hex, substr(u.url,1,75) AS url
FROM visits v JOIN urls u ON v.url = u.id
WHERE v.id IN (6,7,8,11,12,13,14,15,16,17,18,19,20,21)
ORDER BY v.visit_time;
vid   visit_utc            secs     hex       url
----  -------------------  -------  --------  ---------------------------------------------------------------------------
6     2022-04-19 13:54:22  0.0      10000000  https://accounts.craigslist.org/login/home
7     2022-04-19 13:54:22  17.5     A0000000  https://accounts.craigslist.org/login?rp=%2Flogin%2Fhome6rt=L
8     2022-04-19 13:54:39  23.4     30000007  https://accounts.craigslist.org/login/home
11    2022-04-19 13:55:05  0.0      10000000  https://post.craigslist.org/c/bal
12    2022-04-19 13:55:05  0.0      80000000  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5
13    2022-04-19 13:55:05  4.3      A0000000  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5?s=type
14    2022-04-19 13:55:10  5.6      30000007  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5?s=cat
15    2022-04-19 13:55:15  177.7    30000007  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5?s=edit
16    2022-04-19 13:58:13  4.4      30000007  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5?s=geoverify
17    2022-04-19 13:58:17  1.6      30000007  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5?s=editimage
18    2022-04-19 13:58:19  3.2      30000007  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5?s=preview
19    2022-04-19 13:58:22  5.0      30000007  https://post.craigslist.org/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5
20    2022-04-19 13:58:27  5.6      30000000  https://post.craigslist.org/manage/7473121658
21    2022-04-19 13:58:31  0.0      30000000  https://baltimore.craigslist.org/hab/d/baltimore-cheaper-than-rx-supplement
sqlite>
```

Fig 1.19 Posting Details of the chrome Activity

The records that support a posting begin at visit 6 (13:54:22 UTC), when the browser reached the Craigslist login page and, at visit 8, submitted a form to the account home. The posting workflow then followed in a single session under one posting key (/k/sHbOUOi_7BG68qUDIbeTbQ/bQTi5): choose type (visit 13), category (14), create posting (15, open for about 178 seconds), map (16), image (17), preview (18) and confirmation (19), each reached by form submission. Visit 20 shows the management page /manage/7473121658, and visit 21 shows the live advert, “cheaper than Rx supplements”,

under the same listing number. Gmail threads with this title as their subject were opened from 14:04:12 UTC, and ?compose= addresses at visits 44, 45, 72 and 73 show that reply windows were opened.
Together, these record IDs, URLs, and transition timestamps provide a complete chain of evidence establishing that the classified advertisement was created, configured, and published from this browser profile.

The figure below shows a search for pages whose title contains “cheaper than Rx”, the title of the advert identified earlier. I joined the visits and urls tables and flagged any address containing a compose= parameter. The query returned nine visits between 13:58:31 and 15:02:56 UTC. The first, visit 21, is the live Craigslist advert. The other eight are Gmail threads that carry the same title in their subject, which links the advert to an email conversation. Visits 44, 45, 72 and 73 contain compose parameters, which shows that reply windows were opened inside these threads. The open_v column shows they were opened from the thread itself (visits 43 to 46 and 48 to 74). From visit 48 onward, the subject also contains a transaction ID (txid), which I examine in the next section. These records show a link between the advert and the emails, but not what was written or whether anything was sent.

```
┌──(kali㉿kali)-[~/SBT-DF204-CS2]
└─$ sqlite3 working/History_working.db
-- Loading resources from /home/kali/.sqliterc
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> SELECT v.id AS vid, u.id AS uid, datetime(v.visit_time/1000000 - 11644473600,'unixepoch') AS visit_utc,
        v.opener_visit AS open_v, substr(u.title,1,45) AS title,
        CASE WHEN u.url LIKE '%compose=%' THEN 'compose' ELSE '' END AS compose
FROM visits v JOIN urls u ON v.url = u.id
WHERE u.title LIKE '%cheaper than Rx%'
ORDER BY v.visit_time;
```

vid	uid	visit_utc	open_v	title	compose
21	16	2022-04-19 13:58:31	20	cheaper than Rx supplements - health and beau	
43	34	2022-04-19 14:04:12	42	cheaper than Rx supplements - order - unsub.f	
44	35	2022-04-19 14:04:35	43	cheaper than Rx supplements - order - unsub.f	compose
45	36	2022-04-19 14:04:46	44	cheaper than Rx supplements - order - unsub.f	compose
46	34	2022-04-19 14:04:52	45	cheaper than Rx supplements - order - unsub.f	
48	37	2022-04-19 14:56:47	47	cheaper than Rx supplements -- txid: 517b2156	
72	56	2022-04-19 15:02:32	48	cheaper than Rx supplements - order - unsub.f	compose
73	57	2022-04-19 15:02:44	72	cheaper than Rx supplements - order - unsub.f	compose
74	37	2022-04-19 15:02:56	73	cheaper than Rx supplements -- txid: 517b2156	

```
sqlite>
```

Fig 1.20 More Details by filtering “compose” and “cheaper than Rx”

COMMUNICATION, DOWNLOADS AND PAYMENT ACTIVITY

1. Communication

After creating and viewing the advert (visit 21, 13:58:31), the same browser moved to email just seven seconds later. At 13:58:38 (visit 22), a search for “gmail” was entered in the address bar, and from 13:58:40 the browser passed through a chain of redirects and Google sign-in pages (visits 24 to 36). At 13:59:26 (visit 36) it reached the password challenge page (accounts.google.com/signin/v2/challenge/pw), and a cookie-check chain (visits 37 to 41) ended in the inbox at 13:59:31 (visit 42). The page title shows the account as unsub.fscs@gmail.com.

```
sqlite> .headers on
sqlite> .mode column
sqlite> SELECT v.id AS vid, datetime(v.visit_time/1000000 - 11644473600,'unixepoch') AS visit_utc,
       v.from_visit AS from_v, v.opener_visit AS open_v, printf('%X', v.transition & 4294967295) AS hex,
       substr(u.url,1,40) AS url
FROM visits v JOIN urls u ON v.url = u.id WHERE v.id BETWEEN 22 AND 42 ORDER BY v.visit_time;
vid   visit_utc            from_v  open_v  hex       url
----  -------------------  ------  ------  --------  ----------------------------------------
22    2022-04-19 13:58:38  0       0       32000005  https://www.google.com/search?q=gmail&rl
23    2022-04-19 13:58:39  0       22      30000000  https://www.google.com/search?q=gmail&rl
24    2022-04-19 13:58:40  23      0       10000000  https://www.google.com/gmail/
25    2022-04-19 13:58:40  24      0       80000000  https://mail.google.com/mail/
26    2022-04-19 13:58:40  25      0       80000000  https://mail.google.com/mail/u/0/
27    2022-04-19 13:58:40  26      0       80000000  https://accounts.google.com/ServiceLogin
28    2022-04-19 13:58:40  27      0       80000000  https://mail.google.com/intl/en/mail/hel
29    2022-04-19 13:58:40  28      0       80000000  https://www.google.com/intl/en/mail/help
30    2022-04-19 13:58:40  29      0       A0000000  https://www.google.com/gmail/about/
31    2022-04-19 13:58:43  0       0       10000000  https://accounts.google.com/AccountChoos
32    2022-04-19 13:58:43  31      0       80000000  https://accounts.google.com/AccountChoos
33    2022-04-19 13:58:43  32      0       A0000000  https://accounts.google.com/ServiceLogin
34    2022-04-19 13:58:44  0       33      30000000  https://accounts.google.com/ServiceLogin
35    2022-04-19 13:58:44  0       34      30000000  https://accounts.google.com/signin/v2/id
36    2022-04-19 13:59:26  0       35      30000000  https://accounts.google.com/signin/v2/ch
37    2022-04-19 13:59:29  36      0       10000000  https://accounts.google.com/CheckCookie?
38    2022-04-19 13:59:29  37      0       80000000  https://mail.google.com/accounts/SetOSID
39    2022-04-19 13:59:29  38      0       80000000  https://accounts.youtube.com/accounts/Se
40    2022-04-19 13:59:29  39      0       80000000  https://mail.google.com/mail/
41    2022-04-19 13:59:29  40      0       80000000  https://mail.google.com/mail/u/0/
42    2022-04-19 13:59:31  41      41      60000000  https://mail.google.com/mail/u/0/#inbox
sqlite>
```

Fig 1.21 Communication evidence

At 14:04:12 (visit 43), a Gmail thread titled “cheaper than Rx supplements - order” was opened, the same title as the advert, 5 minutes 41 seconds after the advert page was visited. Two reply windows followed (visits 44 and 45, at 14:04:35 and 14:04:46, identified by ?compose= in the address), and the browser returned to the thread at 14:04:52 (visit 46). After a gap in recorded activity of about 50 minutes, a thread with the subject “cheaper than Rx supplements with a txid: 517b2156…afdc5” was opened at 14:56:47 (visit 48). Two more reply windows were opened at 15:02:32 and 15:02:44 (visits 72 and 73), and the browser returned to that thread at 15:02:56 (visit 74).

2. The Imgur link and the download

Five seconds after the transaction-ID thread was opened, at 14:56:52 (visit 49), the browser opened a Google link-wrapper address carrying source=gmail and pointing to imgur.com/dTgrkP7. Its opener_visit is 48, so the link was followed from that email thread. It redirected at once to the Imgur page (visit 50), which stayed open for 18.9 seconds.

```
sqlite> SELECT v.id AS vid, datetime(v.visit_time/1000000 - 11644473600,'unixepoch') AS visit_utc,
       v.from_visit AS from_v, v.opener_visit AS open_v, printf('%X', v.transition & 4294967295) AS hex,
       substr(u.url,1,60) AS url
FROM visits v JOIN urls u ON v.url = u.id WHERE v.id BETWEEN 47 AND 50 ORDER BY v.visit_time;
vid   visit_utc            from_v  open_v  hex       url
----  -------------------  ------  ------  --------  ------------------------------------------------------------
47    2022-04-19 14:05:59  0       46      30000000  https://mail.google.com/mail/u/0/#inbox
48    2022-04-19 14:56:47  0       47      10000000  https://mail.google.com/mail/u/0/#inbox/WhctKKXXDrfCqvtRjhGd
49    2022-04-19 14:56:52  0       48      10000000  https://www.google.com/url?q=https://imgur.com/dTgrkP7&sourc
50    2022-04-19 14:56:52  49      0       A0000000  https://imgur.com/dTgrkP7
sqlite>
```

Fig 1.22 Download Evidence

Concerning the downloaded file, the downloads table contains one record. A file named proof_of_payment.png was saved to C:\Users\FSCS_User\Desktop\. The download started at 14:56:58, six seconds after the Imgur page began loading, and finished at 14:57:10, about 12 seconds later. It received 33,844 bytes out of 33,844, with a completed state, no interruption and no danger flag. The type was image/png, the referrer was https://imgur.com/, and the source tab was https://imgur.com/dTgrkP7.

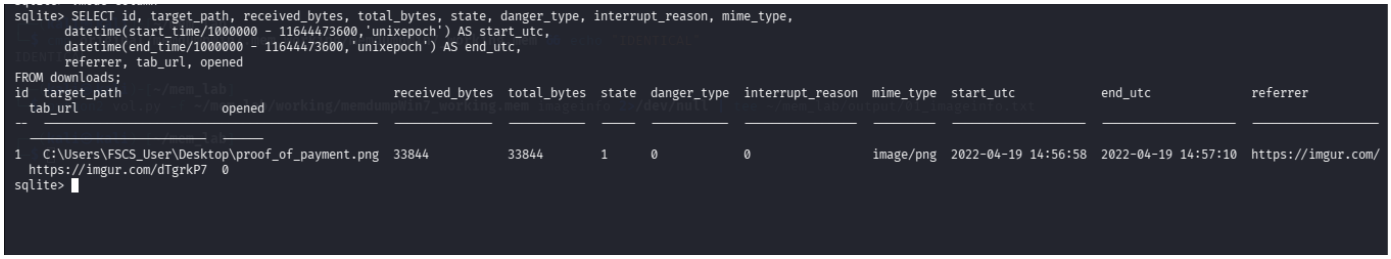


Fig 1.23 Downloaded Evidence and Folder in the Desktop

I also noticed that the source image was named dTgrkP7.png on Imgur but was saved as proof_of_payment.png, so the file name was changed when it was saved. The history does not hold the image itself, a file hash (the hash field is empty) or any record that it was opened (opened is 0), so I can't say what the picture shows or whether it truly shows a payment. The file name is a lead, not proof.

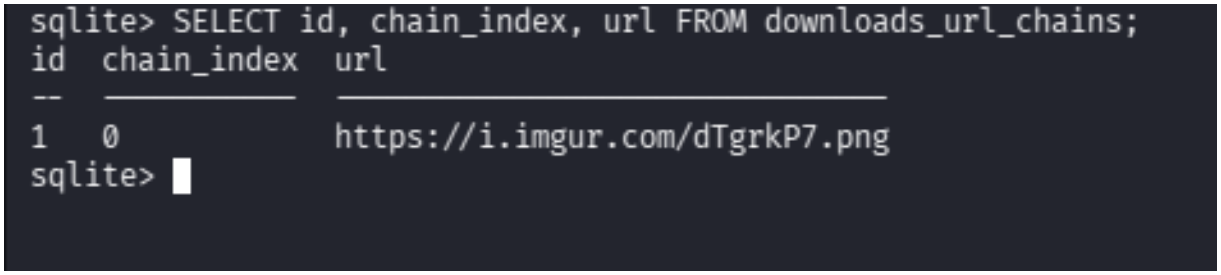


Fig 1.24 Downloaded file from the site imgur.com/

3. Payment verification

About two minutes after the download finished, at 14:58:59 (visit 51), a search for "blockchain explorer" was entered. At 14:59:06 (visit 55) the explorer's search was used with the transaction ID 517b2156…afdc5, which is the same ID that appears in the email subject. The search redirected to the transaction page (visit 56). At 14:59:21 (visit 58), a Bitcoin address page was opened (38RcsURWCDmCbYocmsZCnGz1FtD7x477mt), with opener_visit 57, the transaction page.

```
sqlite> SELECT v.id AS vid, datetime(v.visit_time/1000000 - 11644473600,'unixepoch') AS visit_utc,
       v.from_visit AS from_v, v.opener_visit AS open_v, printf('%X', v.transition & 4294967295) AS hex,
       substr(u.title,1,40) AS title
FROM visits v JOIN urls u ON v.url = u.id WHERE v.id BETWEEN 51 AND 58 ORDER BY v.visit_time;
vid  visit_utc            from_v  open_v  hex       title
---  -------------------  ------  ------  --------  ----------------------------------------
51   2022-04-19 14:58:59  0       0       32000005  blockchain explorer - Google Search
52   2022-04-19 14:59:00  0       51      30000000  blockchain explorer - Google Search
53   2022-04-19 14:59:01  52      0       30000000  Blockchain Explorer - Search the Blockch
54   2022-04-19 14:59:02  0       53      30000000  Blockchain Explorer - Search the Blockch
55   2022-04-19 14:59:06  54      0       10000000  Transaction: 517b2156914944339a96137ad89
56   2022-04-19 14:59:06  55      0       A0000000  Transaction: 517b2156914944339a96137ad89
57   2022-04-19 14:59:06  54      56      30000000  Transaction: 517b2156914944339a96137ad89
58   2022-04-19 14:59:21  54      57      30000000  Address: 38RcsURWCDmCbYocmsZCnGz1FtD7×47
sqlite>
```

Fig 1.25 Payment Details Tracked

```
sqlite> SELECT id, substr(title,1,45) AS title, substr(url,1,60) AS url FROM urls
WHERE url LIKE '%517b2156914944339a96137ad8978408ea52b2fc144c98d3b0b16b21888afdc5%'
   OR title LIKE '%517b2156914944339a96137ad8978408ea52b2fc144c98d3b0b16b21888afdc5%';
id  title                                          url
--  ---------------------------------------------  ------------------------------------------------------------
37  cheaper than Rx supplements -- txid: 517b2156  https://mail.google.com/mail/u/0/#inbox/WhctKKXXDrfCqvtRjhGd
42  Transaction: 517b2156914944339a96137ad8978408  https://www.blockchain.com/search?search=517b2156914944339a9
43  Transaction: 517b2156914944339a96137ad8978408  https://www.blockchain.com/btc/tx/517b2156914944339a96137ad8
51  Transaction: 517b2156914944339a96137ad8978408  https://www.kraken.com/redirect?url=https%3A%2F%2Fmempool.sp
52  Transaction: 517b2156914944339a96137ad8978408  http://www.kraken.com/redirect?t=-_F22IhwssFD
53  Transaction: 517b2156914944339a96137ad8978408  https://www.kraken.com/redirect?t=-_F22IhwssFD
54  Transaction: 517b2156914944339a96137ad8978408  https://mempool.space/tx/517b2156914944339a96137ad8978408ea5
sqlite>
```

Fig 1.26 Filtering by Transaction ID

```
sqlite> SELECT v.id AS vid, datetime(v.visit_time/1000000 - 11644473600,'unixepoch') AS visit_utc,
       v.from_visit AS from_v, v.opener_visit AS open_v, printf('%X', v.transition & 4294967295) AS hex,
       substr(u.title,1,45) AS title
FROM visits v JOIN urls u ON v.url = u.id WHERE v.id BETWEEN 59 AND 71 ORDER BY v.visit_time;
vid  visit_utc            from_v  open_v  hex       title
---  -------------------  ------  ------  --------  ---------------------------------------------
59   2022-04-19 14:59:38  0       0       32000005  kraken - Google Search
60   2022-04-19 14:59:39  0       59      30000000  kraken - Google Search
61   2022-04-19 14:59:47  60      0       30000000  Sign in to Kraken - Kraken | Buy, Sell and Ma
62   2022-04-19 14:59:47  0       61      30000000  Sign in to Kraken - Kraken | Buy, Sell and Ma
63   2022-04-19 15:00:21  0       62      30000000  Approve new device - Kraken | Buy, Sell and M
64   2022-04-19 15:01:11  63      0       30000000
65   2022-04-19 15:01:12  63      64      30000000  Kraken | Buy, Sell and Margin Trade Bitcoin (
66   2022-04-19 15:01:21  63      65      30000000  41,543.80 USD - Kraken - Funding - Overview
67   2022-04-19 15:01:31  0       66      10000000  Transaction: 517b2156914944339a96137ad8978408
68   2022-04-19 15:01:33  67      0       40000000  Transaction: 517b2156914944339a96137ad8978408
69   2022-04-19 15:01:33  68      0       80000000  Transaction: 517b2156914944339a96137ad8978408
70   2022-04-19 15:01:33  69      0       A0000000  Transaction: 517b2156914944339a96137ad8978408
71   2022-04-19 15:02:05  63      66      30000000  41,563.70 USD - Kraken - History - Ledger
sqlite>
```

Fig 1.27 Transaction Trail Evidence

At 14:59:38 (visit 59), a search for Kraken was entered, followed by the sign-in page (visits 61 and 62, 14:59:47) and an “Approve new device” page (visit 63, 15:00:21). The browser then reached the trade, instant-buy and funding pages (visits 64 to 66, 15:01:11 to 15:01:21). At 15:01:31 (visit 67), a link opened from the funding page (its opener is visit 66) began a four-step redirect chain through kraken.com/redirect, ending at mempool.space/tx/517b2156…afdc5 at 15:01:33 (visit 70). This is the same transaction ID,

and it suggests the transaction was linked from the funding page. At 15:02:05 (visit 71), the account's ledger page was opened.

Stage	UTC timestamp	Visit or URL ID	Artifact	Observed fact	Interpretation and limit
Start of activity	13:54:18	Visit 1 / URL 1	visits, urls	Google search for "craigslist" typed in the address bar (0x30000005)	First recorded activity. Shows a search, not intent.
Login	13:54:39	Visit 8	visits	A form was submitted on the Craigslist login page, after about 17 seconds on it.	This looks like a login, but the username and password are not stored.
Making the ad	13:55:05	Visits 11 to 13	visits, urls	The browser went from Craigslist Baltimore to the "create a post" page and got a posting session.	This is where the ad creation starts.
Making the ad	13:55:10 to 13:58:19	Visits 14 to 18	visits	It went through category, create posting, map, image and preview, each time submitting a form. About 3 minutes were spent on the create-posting page.	This shows the steps taken and the time spent. The ad text and price are not saved.
Ad confirmed	13:58:22	Visit 19	visits	The posting confirmation page appeared.	The main confirmation of the ad been created.
Ad live	13:58:27 to 13:58:31	Visits 20, 21 / URLs 15, 16	visits, urls	The "manage posting" page was opened, then the live ad "cheaper than Rx supplements". Both carry the listing number 7473121658.	They are the same post. What was actually being sold isn't shown.

Stage	UTC timestamp	Visit or URL ID	Artifact	Observed fact	Interpretation and limit
Email sign-in	13:58:38 to 13:59:31	Visits 22 to 42	visits	The browser searched “gmail”, went through Google’s sign-in and landed in the inbox of unsub.fscs@gmail.com	I was able to discover the sequence of pages visited but what was typed was not discovered.
Email	14:04:12	Visit 43 / URL 34	visits, urls	An email thread called “cheaper than Rx supplements - order” was opened.	It has the same title as the ad and was opened 5 minutes 41 seconds after the ad was viewed. The content and sender are not shown.
Email replies	14:04:35 to 14:04:52	Visits 44 to 46	visits	Two reply windows were opened, then the browser went back to the thread.	This shows replies were started, not that they were sent.
Gap	14:05:59 to 14:56:47	Visits 47, 48	visits	The browser returned to the inbox, then nothing was recorded for about 50 minutes.	This is a gap in recorded page visits. It doesn’t mean the computer was idle.
Transaction email	14:56:47	Visit 48 / URL 37	visits, urls	A thread was opened with the subject “cheaper than Rx supplements -- txid: 517b2156…afdc5”.	The subject contains a transaction ID. The message itself is not stored.
Image link	14:56:52	Visits 49, 50	visits, urls	The browser followed an Imgur link from that email.	The link came from the email and
Download	14:56:58 to 14:57:10	Download 1	downloads, downloads_url_chains	A file called proof_of_payment.png (33,844 bytes) was saved to the Desktop.	*The name differs from the source file name, so it was* renamed when saved. I can’t see the picture, so I can’t say what it shows.

Stage	UTC timestamp	Visit or URL ID	Artifact	Observed fact	Interpretation and limit
Payment check	14:58:59 to 14:59:21	Visits 51 to 58	visits, urls	The same transaction ID was searched on blockchain.com, and then the transaction page and a Bitcoin address page (38Rcs…) were opened.	The matching ID links the email to this lookup. The amount and who owns the address are not shown.
Exchange account	14:59:38 to 15:01:21	Visits 59 to 66	visits, urls	The browser searched “kraken”, signed in, approved a new device and opened the funding page.	This looks like access to an exchange account. The login details aren’t stored, and I don’t read the balance in the page title as a payment.
Transaction check	15:01:31 to 15:01:33	Visits 67 to 70	visits	A link on the funding page took the browser to the same transaction on mempool.space.	This suggests the transaction was linked from the funding page.
Ledger	15:02:05	Visit 71	visits, urls	The Kraken ledger page was opened.	The page content isn’t saved, so I can’t say what it listed.
Back to email	15:02:32 to 15:02:56	Visits 72 to 74	visits	The browser went back to the transaction email and opened two more reply windows. This is the last record.	This is the end of the recorded activity. Again, replies were started, not proven sent.

EVIDENCE LOG Evidence ID	Source record	Finding	Why it matters	SQL or screenshot reference
E01	History file metadata	Original file size 196,608 bytes, SHA-256 hash b991b0fa....	Establishes chain of custody and evidence integrity before analysis.	sha256sum History terminal output
E02	visits / urls	URLs showing posting workflow and post ID 7473121658 titled *cheaper than Rx supplements*.	Proves ad creation, edit duration (177s), and live URL verification.	Query output visit IDs 11–21
E03	visits / urls	Login state and inbox threads for unsub.fscs@gmail.com.	Links the suspect browser profile to buyer communication.	Query output visit IDs 25–46
E04	downloads / visits	Imgur URL visit leading to download of proof_of_payment.png.	Links email receipt link to local desktop file artifact.	Query output visit IDs 48–50
E05	visits / urls	Lookup of TXID 517b2156914... and wallet 38RcsURWCDm....	Connects the illicit ad transaction directly to public BTC blockchain data.	Query output visit IDs 55–58
E06	visits / urls	Kraken login, USD balance check, and Mempool.space confirmation lookup.	Demonstrates exchange account management and live block confirmation tracking.	Query output visit IDs 59–71

CONCLUSION

The forensic examination of the provided Chrome SQLite History database recovered 74 web activity records showing a continuous sequence of browsing activities on April 19, 2022, between 13:54:18 UTC and 15:02:56 UTC.

The records revealed a workflow involving the creation and publication of a Craigslist listing titled *"cheaper than Rx supplements,"* email communications through unsub.fscs@gmail.com, the acquisition of a local payment receipt file named proof_of_payment.png, and repeated verification of the transaction hash 517b2156914944339a96137ad8978408ea52b2fc144c98d3b0b16b21888afdc5 on public blockchain explorers and the Kraken exchange platform.

These findings establish that the recorded browsing activities occurred within the examined browser environment. However, the browser history alone cannot conclusively link these activities to a specific individual, as the possibility of shared workstation access or unauthorized use of the browser session cannot be ruled out.